

## LAB EXPERIMENT – 5

Monitoring Model Explainability and Data Drift using Evidently AI

**Course Outcome:** CO2

### Aim

To monitor data drift, feature drift, model performance, and model explainability using the Evidently AI library in a Google Colab environment.

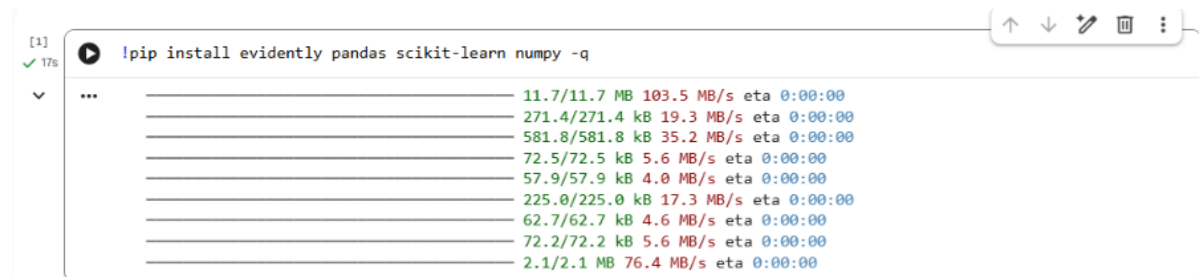
### Step 1: Open Google Colab

Go to <https://colab.research.google.com> and create a new notebook.

### Step 2: Install Required Libraries

Run the following cell:

`!pip install evidently pandas scikit-learn numpy -q`



```
[1] ✓ 17s !pip install evidently pandas scikit-learn numpy -q
...
11.7/11.7 MB 103.5 MB/s eta 0:00:00
271.4/271.4 kB 19.3 MB/s eta 0:00:00
581.8/581.8 kB 35.2 MB/s eta 0:00:00
72.5/72.5 kB 5.6 MB/s eta 0:00:00
57.9/57.9 kB 4.0 MB/s eta 0:00:00
225.0/225.0 kB 17.3 MB/s eta 0:00:00
62.7/62.7 kB 4.6 MB/s eta 0:00:00
72.2/72.2 kB 5.6 MB/s eta 0:00:00
2.1/2.1 MB 76.4 MB/s eta 0:00:00
```

### Step 3: Import Libraries

`import pandas as pd`

`import numpy as np`

`from sklearn.datasets import load_breast_cancer`

`from sklearn.model_selection import train_test_split`

`from sklearn.ensemble import RandomForestClassifier`

`from sklearn.metrics import accuracy_score`

`from evidently import Report`

`from evidently.presets import DataDriftPreset`

`from evidently.presets import ClassificationPreset`

`from evidently.presets import DataSummaryPreset`

```
[3]
✓ Os
import pandas as pd
import numpy as np

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

from evidently import Report
from evidently.presets import DataDriftPreset
from evidently.presets import ClassificationPreset
from evidently.presets import DataSummaryPreset
```

## Step 4: Load Dataset

# Load Breast Cancer dataset

```
data = load_breast_cancer()
```

```
df = pd.DataFrame(data.data, columns=data.feature_names)
```

```
df['target'] = data.target
```

```
print(df.shape)
```

```
df.head()
```

```
[4]
✓ Os
# Load Breast Cancer dataset
data = load_breast_cancer()

df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

print(df.shape)
df.head()
```

... (569, 31)

|   | mean<br>radius | mean<br>texture | mean<br>perimeter | mean<br>area | mean<br>smoothness | mean<br>compactness | mean<br>concavity | mean<br>concave<br>points | mean<br>symmetry | mean<br>fractal<br>dimension | ... | worst<br>texture |
|---|----------------|-----------------|-------------------|--------------|--------------------|---------------------|-------------------|---------------------------|------------------|------------------------------|-----|------------------|
| 0 | 17.99          | 10.38           | 122.80            | 1001.0       | 0.11840            | 0.27760             | 0.3001            | 0.14710                   | 0.2419           | 0.07871                      | ... | 17.33            |
| 1 | 20.57          | 17.77           | 132.90            | 1326.0       | 0.08474            | 0.07864             | 0.0869            | 0.07017                   | 0.1812           | 0.05667                      | ... | 23.41            |
| 2 | 19.69          | 21.25           | 130.00            | 1203.0       | 0.10960            | 0.15990             | 0.1974            | 0.12790                   | 0.2069           | 0.05999                      | ... | 25.53            |
| 3 | 11.42          | 20.38           | 77.58             | 386.1        | 0.14250            | 0.28390             | 0.2414            | 0.10520                   | 0.2597           | 0.09744                      | ... | 26.50            |
| 4 | 20.29          | 14.34           | 135.10            | 1297.0       | 0.10030            | 0.13280             | 0.1980            | 0.10430                   | 0.1809           | 0.05883                      | ... | 16.67            |

5 rows × 31 columns

## Step 5: Split Dataset

```
X = df.drop('target', axis=1)
```

```
y = df['target']
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
```

```
    test_size=0.30,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)  
  
print('Training size:', X_train.shape)  
  
print('Testing size:', X_test.shape)
```

```
[5]  
✓ 0s X = df.drop('target', axis=1)  
y = df['target']  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.30,  
    random_state=42,  
    stratify=y  
)  
  
print('Training size:', X_train.shape)  
print('Testing size:', X_test.shape)  
  
... Training size: (398, 30)  
Testing size: (171, 30)
```

### Step 6: Train Machine Learning Model

```
model = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42  
)  
  
model.fit(X_train, y_train)  
  
y_pred = model.predict(X_test)  
  
acc = accuracy_score(y_test, y_pred)  
  
print('Model Accuracy:', round(acc, 4))
```

```
[6]  
✓ 1s model = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42  
)  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)  
acc = accuracy_score(y_test, y_pred)  
print('Model Accuracy:', round(acc, 4))  
  
... Model Accuracy: 0.9357
```

### Step 7: Create Reference Dataset

```
reference_data = X_train.copy()  
  
reference_data['target'] = y_train.values  
  
reference_data['prediction'] = model.predict(X_train)
```

```
[7]  
✓ 0s reference_data = X_train.copy()  
reference_data['target'] = y_train.values  
reference_data['prediction'] = model.predict(X_train)
```

```
reference_data.head()
```

A Jupyter Notebook cell with the code `reference_data.head()` and its output. The output is a table with 5 rows and 32 columns. The columns are: mean radius, mean texture, mean perimeter, mean area, mean smoothness, mean compactness, mean concavity, mean concave points, mean symmetry, mean fractal dimension, ..., worst perimeter. The rows are indexed 469, 561, 443, 362, and 271.

|     | mean radius | mean texture | mean perimeter | mean area | mean smoothness | mean compactness | mean concavity | mean concave points | mean symmetry | mean fractal dimension | ... | worst perimeter |
|-----|-------------|--------------|----------------|-----------|-----------------|------------------|----------------|---------------------|---------------|------------------------|-----|-----------------|
| 469 | 11.62       | 18.18        | 76.38          | 408.8     | 0.11750         | 0.14830          | 0.10200        | 0.05564             | 0.1957        | 0.07255                | ... | 88.14           |
| 561 | 11.20       | 29.37        | 70.67          | 386.0     | 0.07449         | 0.03558          | 0.00000        | 0.00000             | 0.1060        | 0.05502                | ... | 75.19           |
| 443 | 10.57       | 18.32        | 66.82          | 340.9     | 0.08142         | 0.04462          | 0.01993        | 0.01111             | 0.2372        | 0.05768                | ... | 69.35           |
| 362 | 12.76       | 18.84        | 81.87          | 496.6     | 0.09676         | 0.07952          | 0.02688        | 0.01781             | 0.1759        | 0.06183                | ... | 87.82           |
| 271 | 11.29       | 13.04        | 72.23          | 388.0     | 0.09834         | 0.07608          | 0.03265        | 0.02755             | 0.1769        | 0.06270                | ... | 78.21           |

5 rows x 32 columns

### Step 8: Create Current Dataset

```
current_data = X_test.copy()
```

```
current_data['target'] = y_test.values
```

```
current_data['prediction'] = y_pred
```

```
current_data.head()
```

A Jupyter Notebook cell with the code `current_data = X_test.copy()`, `current_data['target'] = y_test.values`, `current_data['prediction'] = y_pred`, and `current_data.head()`. The output is a table with 5 rows and 32 columns. The columns are: mean radius, mean texture, mean perimeter, mean area, mean smoothness, mean compactness, mean concavity, mean concave points, mean symmetry, mean fractal dimension, ..., worst perimeter. The rows are indexed 14, 150, 275, 83, and 86.

|     | mean radius | mean texture | mean perimeter | mean area | mean smoothness | mean compactness | mean concavity | mean concave points | mean symmetry | mean fractal dimension | ... | worst perimeter |
|-----|-------------|--------------|----------------|-----------|-----------------|------------------|----------------|---------------------|---------------|------------------------|-----|-----------------|
| 14  | 13.73       | 22.61        | 93.60          | 578.3     | 0.11310         | 0.22930          | 0.21280        | 0.08025             | 0.2069        | 0.07682                | ... | 108.8           |
| 150 | 13.00       | 20.78        | 83.51          | 519.4     | 0.11350         | 0.07589          | 0.03136        | 0.02645             | 0.2540        | 0.06087                | ... | 90.8            |
| 275 | 11.89       | 17.36        | 76.20          | 435.6     | 0.12250         | 0.07210          | 0.05929        | 0.07404             | 0.2015        | 0.05875                | ... | 79.4            |
| 83  | 19.10       | 26.29        | 129.10         | 1132.0    | 0.12150         | 0.17910          | 0.19370        | 0.14690             | 0.1634        | 0.07224                | ... | 141.3           |
| 86  | 14.48       | 21.46        | 94.25          | 648.2     | 0.09444         | 0.09947          | 0.12040        | 0.04938             | 0.2075        | 0.05636                | ... | 108.4           |

5 rows x 32 columns

### Step 9: Simulate Data Drift

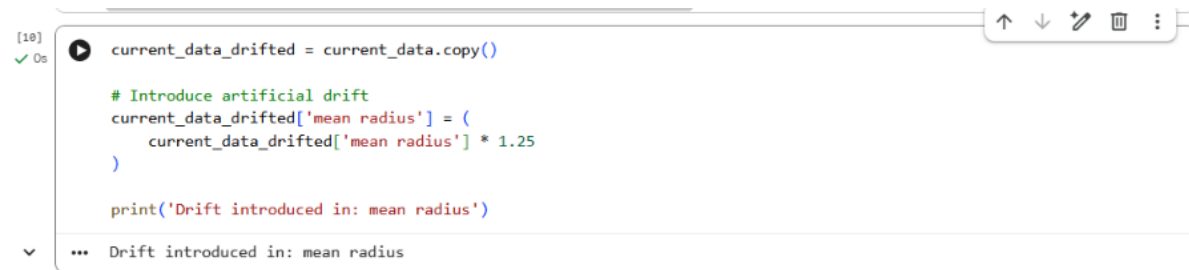
We intentionally change one feature so Evidently can detect drift.

```
current_data_drifted = current_data.copy()
```

```
# Introduce artificial drift
```

```
current_data_drifted['mean radius'] = (  
    current_data_drifted['mean radius'] * 1.25  
)
```

```
print('Drift introduced in: mean radius')
```



```
[10]
✓ 0s
current_data_drifted = current_data.copy()

# Introduce artificial drift
current_data_drifted['mean radius'] = (
    current_data_drifted['mean radius'] * 1.25
)

print('Drift introduced in: mean radius')
```

... Drift introduced in: mean radius

### Step 10: Generate Data Drift Report

```
drift_report = Report(metrics=[
    DataDriftPreset()
])

drift_report.run(
    reference_data=reference_data,
    current_data=current_data_drifted
)

drift_report
```



```
[11]
✓ 3s
drift_report = Report(metrics=[
    DataDriftPreset()
])
drift_report.run(
    reference_data=reference_data,
    current_data=current_data_drifted
)
drift_report
```

... <evidently.core.report.Report at 0x7b2f878f0080>

### Step 11: Generate Data Summary Report

This replaces the old DataQualityPreset.

```
summary_report = Report(metrics=[
    DataSummaryPreset()
])

summary_report.run(
    reference_data=reference_data,
    current_data=current_data_drifted
)

summary_report
```

```
[12] summary_report = Report(metrics=[
✓ 6s DataSummaryPreset()
])
summary_report.run(
    reference_data=reference_data,
    current_data=current_data_drifted
)
summary_report

... <evidently.core.report.Report at 0x7b2f877edaf0>
```

## Step 12: Generate Classification Performance Report

### 12.1 Import Required Classes

from evidently import Report

from evidently.presets import ClassificationPreset

from evidently import BinaryClassification

from evidently import Dataset, DataDefinition

### 12.2 Create Dataset Definition Correctly

```
definition = DataDefinition(
    classification=[
        BinaryClassification(
            target="target",
            prediction_labels="prediction"
        )
    ]
)
```

```
[15]
✓ 0s from evidently import Report
from evidently.presets import ClassificationPreset
from evidently import BinaryClassification
from evidently import Dataset, DataDefinition
```

```
[16]
✓ 0s definition = DataDefinition(
    classification=[
        BinaryClassification(
            target="target",
            prediction_labels="prediction"
        )
    ]
)
```

### 12.3 Create Evidently Datasets

ref\_dataset = Dataset.from\_pandas(

```

reference_data,
data_definition=definition
)
cur_dataset = Dataset.from_pandas(
    current_data_drifted,
    data_definition=definition
)

```

```

[17]
✓ 0s
ref_dataset = Dataset.from_pandas(
    reference_data,
    data_definition=definition
)
cur_dataset = Dataset.from_pandas(
    current_data_drifted,
    data_definition=definition
)

```

#### 12.4. Generate Classification Performance Report

```

classification_report = Report([
    ClassificationPreset()
])
classification_snapshot = classification_report.run(
    reference_data=ref_dataset,
    current_data=cur_dataset
)
classification_snapshot

```

```

classification_report = Report([
    ClassificationPreset()
])
classification_snapshot = classification_report.run(
    reference_data=ref_dataset,
    current_data=cur_dataset
)
classification_snapshot

```

...

| Classification Model Performance. Target: 'target' |              |              |              |  |
|----------------------------------------------------|--------------|--------------|--------------|--|
| Current: Model Quality Metrics                     |              |              |              |  |
| <b>0.936</b>                                       | <b>0.944</b> | <b>0.953</b> | <b>0.949</b> |  |
| Accuracy                                           | Precision    | Recall       | F1           |  |

### 13. Save Classification Report

```

classification_snapshot.save_html(
    "classification_monitoring_report.html"
)

```

```

[19] ✓ Os
classification_snapshot.save_html(
    "classification_monitoring_report.html"
)

```

### Step 14: Check File Creation

```

import os
os.listdir()

```

```

[28] ✓ Os
import os
os.listdir()

['.config', 'classification_monitoring_report.html', 'sample_data']

```

### Step 15: Data Drift Report

```

drift_report = Report([
    DataDriftPreset()
])
drift_snapshot = drift_report.run(
    reference_data=reference_data,
    current_data=current_data_drifted
)

```

```
)  
drift_snapshot.save_html(  
    "data_drift_report.html"  
)
```



A screenshot of a Jupyter Notebook code cell. The cell contains Python code for generating a drift report. To the left of the code, there is a status bar showing '[33]' and a green checkmark with '6s', indicating successful execution. The code is as follows:

```
drift_report = Report([  
    DataDriftPreset()  
)  
  
drift_snapshot = drift_report.run(  
    reference_data=reference_data,  
    current_data=current_data_drifted  
)  
  
drift_snapshot.save_html(  
    "data_drift_report.html"  
)
```

### Step 16: Data Quality Report

```
quality_report = Report([  
    DataSummaryPreset()  
)  
  
quality_snapshot = quality_report.run(  
    reference_data=reference_data,  
    current_data=current_data_drifted  
)  
  
quality_snapshot.save_html(  
    "data_quality_report.html"  
)
```

### Step 17: Classification Performance Monitoring

```
classification_report = Report([  
    ClassificationPreset()  
)  
  
classification_snapshot = classification_report.run(  
    reference_data=ref_dataset,  
    current_data=cur_dataset  
)  
  
classification_snapshot.save_html(  
    "classification_monitoring_report.html"
```

)

```
[35] ✓ Os  classification_report = Report([  
    ClassificationPreset()  
])  
  
classification_snapshot = classification_report.run(  
    reference_data=ref_dataset,  
    current_data=cur_dataset  
)  
  
classification_snapshot.save_html(  
    "classification_monitoring_report.html"  
)
```

### Step 18: Final Download Cell

from google.colab import files

files.download("data\_drift\_report.html")

files.download("data\_quality\_report.html")

files.download("classification\_monitoring\_report.html")

```
[36] ✓ Os  from google.colab import files  
  
files.download("data_drift_report.html")  
files.download("data_quality_report.html")  
files.download("classification_monitoring_report.html")
```